

MacroFit Final Project Report

Nagur Pavan | PE6201 Emerging AI Technologies | 2 October 2026

Executive Summary

MacroFit is a notebook-based meal planner for adults who usually buy prepared food. The user enters a profile, daily budget, and an ordinary request such as “vegetarian lunch and spicy chicken dinner.” Python calculates nutrition targets, filters the menu, and creates valid three-meal plans. GPT-4o-mini interprets flexible language and compares only the plans that Python has already approved. This division gives the model a useful role without allowing it to alter prices, nutrition, or safety rules.

The final OpenRouter evaluation passed 18 of 20 human-written cases, made 25 model calls, used 26,521 tokens, and cost US\$0.00525165. It had no technical errors and no selected plan violated an implemented hard requirement. Two normal requests were openly reported as infeasible. Manual review also found five weak model explanations, which led to a final safeguard: the explanation shown to the user is built from verified menu fields rather than trusted directly from the model.

Why This Architecture Was Used

The project uses a staged architecture because the work contains two different kinds of decisions. Budget totals, calorie limits, dietary categories, meal timing, duplicate checks, and nutrition calculations have exact answers. Natural phrases such as “comfort food,” “something portable,” or “anything for dinner” need interpretation. Combining both tasks inside one model call would make correct arithmetic difficult to audit and would allow fluent output to hide a broken constraint.

The request first passes deterministic checks for attacks, contradictions, and unsafe wording. A language classifier then converts the request into a closed JSON schema. Python applies those fields to the trusted menu and nutrition data, searches possible meal combinations, and produces no more than three distinct valid plans. The model may rank those plans, but it must return one listed plan identifier. Python checks the identifier and the final meal metadata before showing a result. This architecture keeps every important boundary visible in the notebook.

Why an Optimizer Was Used Instead of an Agent

An agent was unnecessary for the core planning problem. The available actions are known in advance: read local JSON, calculate targets, test meal combinations, and select within fixed rules. An optimizer can search these combinations consistently and can explain why no feasible plan exists. An agent would add repeated model calls, variable behaviour, higher cost, and a larger attack surface without improving the arithmetic.

The optimizer also provides the correct baseline for evaluation. Meeting the budget and reaching nutrition thresholds are achievements of Python, not evidence that AI helped. The model earns its place only when it chooses a more appealing option from the same valid shortlist. This follows the course principle of using the smallest suitable tool rather than applying an agent to a bounded calculation problem.

Why the AI Layer Is a Classifier

Layer 1 is a classifier because its task is narrow: translate open-ended food language into predefined fields for breakfast, lunch, dinner, spice, portability, heaviness, cuisine, format, and clarification. It does not browse, calculate, execute tools in a loop, or decide when the task is complete. Those are agent behaviours and are not needed here.

A classifier is easier to test because the expected output is typed and limited. It can recognise meaning that brittle keyword rules miss, including meal boundaries, negation, and subjective wording. Python then re-derives explicit requirements from the original prompt and overrides an incorrect model interpretation before optimization. This protects the plan while still using AI where ordinary rules are weakest.

What Was Built and What Was Bought

I built the profile calculation, data join, optimizer, budget and nutrition checks, meal-slot rules, dietary filters, no-repeat logic, recommendation history, intake recalculation, guardrails, schemas, final validation, evaluation cases, and cost logging. These components require control, repeatability, and inspection. They are inexpensive to run locally and can be tested without an API.

I bought access to language intelligence through OpenRouter. Training or hosting a foundation model would be unjustified for a prototype with 60 menu items. Renting keeps the starting cost low and allows the model to be replaced without rewriting the optimizer. Only the user's request and a compact shortlist are sent to the provider; the full dataset and persistent history are not sent. The trade-off is dependence on provider availability, latency, pricing, and model behaviour.

Why GPT-4o-mini Was Selected

GPT-4o-mini was selected because it completed the project's structured JSON calls reliably at low cost. Earlier Qwen trials encountered shared-provider rate limits and JSON response-format problems. A larger GPT model would cost more without a demonstrated need for this short classification and ranking task. Claude remained a possible alternative, but the project did not have comparative evidence that it would improve the outcome. OpenRouter keeps the model replaceable, while the recorded evaluation names the exact model used so the result can be reproduced.

Outcome and Evaluation Critique

Today, a person who buys every meal must compare menus, prices, nutrition, and preferences separately. MacroFit demonstrates one flow that turns those inputs into a complete day and recalculates the remaining meals after confirmed intake. Its contribution is the decision process rather than a claim that synthetic nutrition is ready for public use.

The 20 evaluation cases included normal requests, meal-specific wording, negation, contradictions, an impossible budget, prompt injection, unclear references, and an extreme intake conflict. E05 asked for a hearty vegetarian dinner that the dataset did not contain. E07 asked for portable, non-soupy food that could not meet both the budget and nutrition thresholds. Both cases failed visibly instead of returning unsuitable food. The 11 successful Layer 2 recommendations give an evaluation cost of about US\$0.00048 each, but this excludes product hosting and human review.

The 90% pass rate is useful evidence, but the sample is small and written by the project author. It tests routing and constraint handling better than real user satisfaction. The saved run also found five raw explanation issues: the chosen meals were valid, but the model sometimes mentioned an irrelevant trade-off or claimed a quality, such as warmth or comfort, that the dataset could not verify. The raw explanations remain in the evaluation file for audit. They are not used as the final user-facing reason. A larger blind comparison is still needed to show whether people prefer the model's choice to the optimizer's choice.

Difficulties Performance Tuning and Rough Edges

The hardest problem was keeping subjective language attached to the correct meal. Earlier rules confused "veg" inside "non-veg," ignored negation, and carried a lunch preference into dinner. Meal-aware parsing, explicit conflict checks, and final metadata validation fixed these errors. Near-duplicate shortlists were reduced by deduplicating item sets. Repeated recommendations were limited with session history and

recently eaten penalties. Payloads were shortened to three plans, unnecessary Layer 2 calls were skipped, and malformed model output receives one controlled retry.

The remaining weaknesses are clear. Prices and nutrition are synthetic; several nutrition matches have low confidence; the dataset cannot verify comfort, warmth, allergies, or personal familiarity; and history disappears when Colab restarts. Real deployment would require verified outlet data, allergy review, persistent user-controlled history, latency measurement, and more independent blind judgements. Until then, MacroFit should be presented as a tested decision-support prototype, not a nutrition or medical service.